ASSIGNMENT-18.4

Name: Chirra Jyothika

Ht.no:2303A51280

Batch:05

Task 1- Movie Database API Integration

Scenario

You are building a movie recommendation tool that fetches movie details from a public Movie Database API.

Task

Use AI to generate a Python script that connects to a Movie API and retrieves movie information by title.

Instructions

• Use requests library to fetch movie details.

• Accept movie title as user input.

• Display:

o Title

o Release year

o Rating

o Plot summary

• Implement error handling for:

o Invalid movie name

o API key errors

o Network timeout

o Empty API response

Prompt:

```
Create a Python script using the requests library to fetch movie
details from OMDb API.Requirements: Take movie title as user input, Fetch
data from API using API key,Display: Title, Year, IMDb Rating, Plot,Handle
errors: invalid movie, API key issues, network errors, empty response,Use clean
formatting and functions
```

Code:

```
fetch_movie_details.py > fetch_movie_details > e
import requests

def fetch_movie_details(title, api_key):
    url = "http://www.omdbapi.com/"
    params = {
        "t": title,
        "apikey": api_key
    }
    try:
        response = requests.get(url, params=params, timeout=5)
        response.raise_for_status()
        data = response.json()

        if not data or data.get("Response") == "False":
            error = data.get("Error", "Unknown error") if data else "Empty response from API."
            if error == "Movie not found!":
                print(f"❌ Movie '{title}' not found.")
            elif "API key" in error:
                print(f"❌ API Key Error: {error}")
            else:
                print(f"❌ Error: {error}")
            return

        print("\n🎬 Movie Details")
        print("-" * 40)
        print(f"Title       : {data.get('Title', 'N/A')}")
        print(f"Year        : {data.get('Year', 'N/A')}")
        print(f"IMDb Rating : {data.get('imdbRating', 'N/A')}")
        print(f"Plot        : {data.get('Plot', 'N/A')}")
        print("-" * 40)

    except requests.exceptions.Timeout:
        print("❌ Network timeout. Please try again later.")
    except requests.exceptions.RequestException as e:
        print(f"❌ Network error: {e}")
    except Exception as e:
        print(f"❌ Unexpected error: {e}")

def main():
    api_key = "51095f30"  # Your OMDb API key
    title = input("Enter movie title: ").strip()
    if not title:
        print("❌ Movie title cannot be empty.")
        return
    fetch_movie_details(title, api_key)

if __name__ == "__main__":
    main()
```

Output:

```
PS C:\Users\maddi\OneDrive\Desktop\AI ASSISTED CODING> & C:/Users/maddi/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/maddi/OneDrive/Desktop/AI ASSISTED CODING/fetch_movie_details.py"
Enter movie title: Guardians of the Galaxy Vol. 2

🎬 Movie Details
----------------------------------------
Title       : Guardians of the Galaxy Vol. 2
Year        : 2017
IMDb Rating : 7.6
Plot        : The Guardians struggle to keep together as a team while dealing with their personal family issues, notably Star-Lord's encounter with his father, the ambitious celestial being Ego.
----------------------------------------
```

Explanation:

- Used API integration to fetch real-time movie data
- Sent HTTP request using requests.get()

- Parsed JSON response to extract required fields
- Displayed formatted output for better readability
- Implemented robust error handling to avoid crashes

Task 2: Public Transport Schedule API

Scenario

A city transport app needs to fetch bus/train arrival timings from a

transport API.

Task

Use AI assistance to integrate a transport schedule API.

Instructions

• Fetch arrival times for a given station/stop ID.

• Validate user input.

• Handle errors such as:

o Invalid stop ID

o Server unavailable

o Timeout errors

• Display next 3 arrival times in formatted output.

• Implement retry logic (minimum 1 retry attempt).

Prompt:

```
Provide a Python program that fetches public transport
arrival timings from an API using a given stop ID. The
program should validate user input, handle errors such as
invalid stop ID, server unavailability, and timeout issues using
try-except blocks, and implement a retry mechanism with at
least one retry attempt. The output should display the next
three arrival times in a clean and formatted manner.
```

Code:

```python
import requests
import time

API_URL = "https://api.tfl.gov.uk/StopPoint/{stop_id}/Arrivals"
def fetch_arrival_times(stop_id):
    url = API_URL.format(stop_id=stop_id)
    try:
        response = requests.get(url, timeout=5)
        response.raise_for_status()
        data = response.json()
        if not isinstance(data, list) or not data:
            print("No arrival times found for this stop ID.")
            return None
        # Sort by expected arrival time
        data.sort(key=lambda x: x.get("expectedArrival", ""))
        arrivals = []
        for item in data[:3]:
            line = item.get("lineName", "?")
            destination = item.get("destinationName", "?")
            time_str = item.get("expectedArrival", "?")
            arrivals.append(f"Line {line} to {destination} at {time_str}")
        return arrivals
    except requests.exceptions.HTTPError as e:
        if response.status_code == 404:
            print("Invalid stop ID. Please check and try again.")
        else:
            print(f"HTTP error occurred: {e}")
    except requests.exceptions.Timeout:
        print("Request timed out. Retrying...")
        return "timeout"
    except requests.exceptions.ConnectionError:
        print("Server unavailable. Please check your connection.")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")
    return None

def main():
    stop_id = input("Enter the stop ID: ").strip()
    if not stop_id.isalnum():
        print("Invalid input. Stop ID should be alphanumeric.")
        return

    for attempt in range(2):  # 1 retry
        result = fetch_arrival_times(stop_id)
        if result == "timeout":
            time.sleep(2)
            continue
        elif result:
            print("\nNext 3 Arrival Times:")
            for i, arrival in enumerate(result, 1):
                print(f"{i}. {arrival}")
            break
        else:
            break

if __name__ == "__main__":
    main()
```

Output:

```
Enter the stop ID: 490008660N

Next 3 Arrival Times:
1. Line 214 to Highgate Village at 2026-04-01T13:14:00Z
2. Line 88 to Parliament Hill Fields at 2026-04-01T13:15:05Z
3. Line 214 to Highgate Village at 2026-04-01T13:22:22Z
Next 3 Arrival Times:
1. Line 214 to Highgate Village at 2026-04-01T13:14:00Z
2. Line 88 to Parliament Hill Fields at 2026-04-01T13:15:05Z
3. Line 214 to Highgate Village at 2026-04-01T13:22:22Z
2. Line 88 to Parliament Hill Fields at 2026-04-01T13:15:05Z
3. Line 214 to Highgate Village at 2026-04-01T13:22:22Z
PS C:\Users\maddi\OneDrive\Desktop\AI ASSISTED CODING>
```

Explanation:

- requests.get() → fetch API data
- timeout=5 → prevents hanging
- sort() → sorts arrivals by time
- try-except → handles runtime errors
- Retry loop ensures reliability

Task 3 - Stock Market Data API

Scenario

You are developing a finance dashboard that retrieves live stock prices.

Task

Use AI to generate a Python script that fetches stock data from a

financial API.

Instructions

• Accept stock symbol as input.

• Retrieve:

o Current price

o Day high

o Day low

• Handle:

o Invalid stock symbol

o API rate limits

o Missing fields in JSON response

• Validate and safely parse JSON response.

Expected Output

• Script fetches and displays stock information.

• No unhandled exceptions.

• Clear error messages for invalid symbols or rate limits.

Prompt:

```
Provide a Python script that fetches stock market data using
a financial API. The program should accept a stock symbol as
input and retrieve the current price, day high, and day low. It
must handle errors such as invalid stock symbols, API rate
limits, and missing fields in the JSON response. Ensure
proper validation and safe parsing of the API response using
try-except blocks, and display clear error messages without
crashing.
```

Code:

```python
import requests

API_KEY = "1J5HMI3Z6T6Q4D3K"  # Replace with your Alpha Vantage API key
API_URL = "https://www.alphavantage.co/query"

def fetch_stock_data(symbol):
    params = {
        "function": "GLOBAL_QUOTE",
        "symbol": symbol,
        "apikey": API_KEY
    }
    try:
        response = requests.get(API_URL, params=params, timeout=10)
        response.raise_for_status()
        data = response.json()
        if "Note" in data:
            print("API rate limit reached. Please try again later.")
            return
        if "Global Quote" not in data or not data["Global Quote"]:
            print("Invalid stock symbol or no data available.")
            return
        quote = data["Global Quote"]
        try:
            price = quote["05. price"]
            high = quote["03. high"]
            low = quote["04. low"]
        except KeyError as e:
            print(f"Missing field in API response: {e}")
            return
        print(f"\nStock: {symbol.upper()}")
        print(f"Current Price: {price}")
        print(f"Day High: {high}")
        print(f"Day Low: {low}")
    except requests.exceptions.Timeout:
        print("Request timed out. Please check your connection and try again.")
    except requests.exceptions.RequestException as e:
        print(f"Network or API error: {e}")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")

def main():
    symbol = input("Enter stock symbol (e.g., AAPL): ").strip().upper()
    if not symbol.isalnum():
        print("Invalid input. Stock symbol should be alphanumeric.")
        return
    fetch_stock_data(symbol)

if __name__ == "__main__":
    main()
```

Output:

```
Enter stock symbol (e.g., AAPL): GOOGL

Stock: GOOGL
Current Price: 287.5600
Day High: 288.0800
Day Low: 277.0900
PS C:\Users\maddi\OneDrive\Desktop\AI ASSISTED CODING>
```

Explanation:

- requests.get() → fetch API data
- response.json() → parse JSON
- "Global Quote" → contains stock info
- .get() → prevents KeyError
- try-except → ensures no crashes

Task 4: Geolocation API (IP AddressLookup)

Scenario

A cybersecurity tool needs to detect the geographical location of a user based on IP address.

Task

Integrate a Geolocation API using AI assistance.

Instructions

• Accept IP address as input.

• Fetch:

o Country

o City

o ISP

• Validate IP format before making API call.

• Handle:

o Invalid IP format

o API failure

o JSON decoding errors

Prompt:

```
Provide a Python script that accepts an IP address as input
and retrieves geolocation details such as country, city, and
ISP using a public API. The program should validate the IP
address format before making the API call and handle errors
including invalid IP format, API failure, and JSON decoding
issues. Ensure robust error handling using try-except blocks
and display clean, formatted output.
```

Code:

```python
import requests
import re

def is_valid_ip(ip):
    # Simple IPv4 validation
    pattern = re.compile(r"^(?:[0-9]{1,3}\.){3}[0-9]{1,3}$")
    if not pattern.match(ip):
        return False
    parts = ip.split('.')
    return all(0 <= int(part) <= 255 for part in parts)

def fetch_geolocation(ip):
    url = f"http://ip-api.com/json/{ip}"
    try:
        response = requests.get(url, timeout=5)
        response.raise_for_status()
        try:
            data = response.json()
        except Exception:
            print("Error: Failed to decode JSON response.")
            return
        if data.get("status") != "success":
            print(f"API Error: {data.get('message', 'Unknown error')}")
            return
        country = data.get("country", "N/A")
        city = data.get("city", "N/A")
        isp = data.get("isp", "N/A")
        print(f"\nIP Address: {ip}")
        print(f"Country: {country}")
        print(f"City: {city}")
        print(f"ISP: {isp}")
    except requests.exceptions.Timeout:
        print("Request timed out. Please try again.")
    except requests.exceptions.RequestException as e:
        print(f"Network or API error: {e}")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")

def main():
    ip = input("Enter an IP address: ").strip()
    if not is_valid_ip(ip):
        print("Invalid IP address format.")
        return
    fetch_geolocation(ip)

if __name__ == "__main__":
    main()
```

Output:

```
Enter an IP address: 93.184.216.34

IP Address: 93.184.216.34
Country: Poland
City: Warsaw
PS C:\Users\maddi\OneDrive\Desktop\AI ASSISTED COD
Enter an IP address: 93.184.216.34

IP Address: 93.184.216.34
Country: Poland
City: Warsaw
IP Address: 93.184.216.34
Country: Poland
City: Warsaw
ISP: EDGECAST
```

Explanation:

- requests.get() → fetch API data
- regex → validates IP format
- .get() → safe dictionary access
- try-except → handles runtime errors

Task 5: Real-Time Application: Payment Gateway Simulation API

Scenario

You are building a test payment integration system that connects to a sandbox payment API.

Task

Use AI to generate a Python script that simulates payment processing.

Instructions

• Send POST request with:

o Amount

o Currency

o Payment method

• Validate input values.

• Handle:

o Invalid payment details

o HTTP status errors (400, 401, 500)

o Network interruptions

• Display transaction status clearly.

• Log failed transactions to a file.

Prompt:

```
Provide a Python script that simulates a payment gateway
using a sandbox API. The program should send a POST
request with amount, currency, and payment method. It
must validate inputs and handle errors such as invalid
payment details, HTTP errors (400, 401, 500), and network
issues. The script should display transaction status clearly
and log failed transactions into a file using proper exception
handling.
```

Code:

```python
import requests
import logging

# 🔹 Configure logging
logging.basicConfig(filename='failed_transactions.log',
                    level=logging.ERROR,
                    format='%(asctime)s - %(message)s')

def validate_input(amount, currency, method):
    if amount <= 0:
        return False, "Amount must be greater than 0"
    if currency not in ["USD", "EUR", "INR"]:
        return False, "Unsupported currency"
    if method.lower() not in ["card", "upi", "netbanking"]:
        return False, "Invalid payment method"
    return True, ""

def process_payment(amount, currency, method):
    url = "https://jsonplaceholder.typicode.com/posts"  # Mock API

    payload = {
        "amount": amount,
        "currency": currency,
        "method": method
    }

    try:
        response = requests.post(url, json=payload, timeout=5)

        # Handle HTTP errors
        if response.status_code == 400:
            raise Exception("Bad Request (400)")
        elif response.status_code == 401:
            raise Exception("Unauthorized (401)")
        elif response.status_code >= 500:
            raise Exception("Server Error (500)")
```

```python
        # Handle HTTP errors
        if response.status_code == 400:
            raise Exception("Bad Request (400)")
        elif response.status_code == 401:
            raise Exception("Unauthorized (401)")
        elif response.status_code >= 500:
            raise Exception("Server Error (500)")

        if response.status_code == 201:
            print("\n✅ Payment Successful!")
            print(f"Amount: {amount} {currency}")
            print(f"Method: {method}")
        else:
            raise Exception(f"Unexpected status code: {response.status_code}")

    except requests.exceptions.Timeout:
        logging.error("Timeout error during transaction")
        print("❌ Error: Request timed out.")

    except requests.exceptions.ConnectionError:
        logging.error("Network connection error")
        print("❌ Error: Network issue.")

    except Exception as e:
        logging.error(f"Transaction failed: {str(e)}")
        print(f"❌ Payment Failed: {str(e)}")


# 🔹 Input Handling
try:
    amount = float(input("Enter amount: "))
    currency = input("Enter currency (USD/INR/EUR): ").upper()
    method = input("Enter payment method (card/upi/netbanking): ")

    valid, message = validate_input(amount, currency, method)

    if not valid:
        print("❌ Input Error:", message)
    else:
        process_payment(amount, currency, method)

except ValueError:
    print("❌ Error: Invalid input format.")
```

Output:

```
Enter amount: 100
Enter currency (USD/INR/EUR): usd
Enter payment method (card/upi/netbanking): card

✅ Payment Successful!
Amount: 100.0 USD
Method: card
PS C:\Users\maddi\OneDrive\Desktop\AI ASSISTED CODING> & C:/Users/mad
Enter amount: 120
Enter currency (USD/INR/EUR): eur
Enter payment method (card/upi/netbanking): netbanking

✅ Payment Successful!
Amount: 120.0 EUR
Method: netbanking
PS C:\Users\maddi\OneDrive\Desktop\AI ASSISTED CODING>
```

Explanation:

- requests.post() → sends payment data
- json=payload → sends JSON body
- logging → records failed transactions
- try-except → prevents crashes
- Status codes → used to detect errors